



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE TIJUANA

**SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN**

SEMESTRE:

Agosto-Diciembre 2025

MATERIA:

Patrones de Diseño de Software

TÍTULO ACTIVIDAD:

Examen 2

UNIDAD A EVALUAR:

Unidad 2

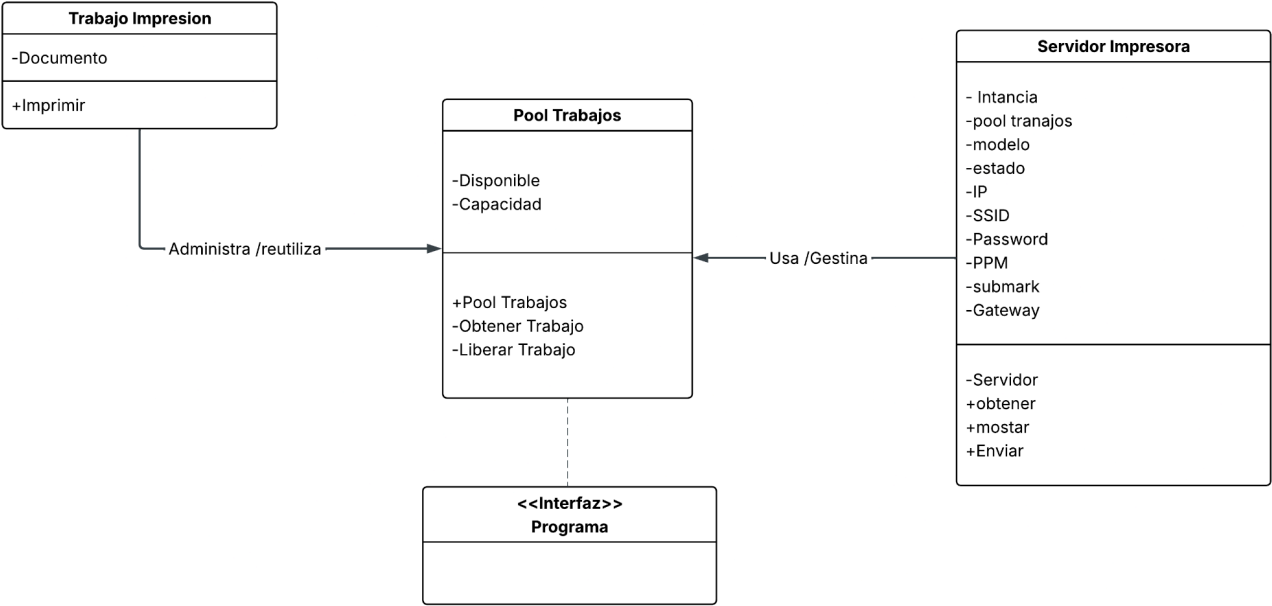
NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

Liczi Citlali Flores Ramirez - 21210537

NOMBRE DEL MAESTRO (A):

MARIBEL GUERRERO LUIS

Diagrama UML



Código

Aquí se llaman dos veces al método *ObtenerInstancia()* del patrón Singleton.

-Pero el Singleton solo crea una única instancia, así que aunque se llame dos veces, ambas variables apuntan al mismo servidor.

- Esto demuestra que el patrón está funcionando correctamente: no se crean dos impresoras, solo una que todos comparten.

- *Mostrar Estado Impresora* Este método muestra en pantalla información sobre la impresora: Modelo, Velocidad, Estado (encendida o apagada) etc.

```
9      {
10     }
11     0 referencias
12     internal class Program
13     {
14     }
15     0 referencias
16     static void Main(string[] args)
17     {
18     {
19         Console.WriteLine("  GESTIÓN DE COLAS DE IMPRESIÓN EN RED : \n");
20
21         // 1 Obtener la única instancia del servidor para que no crea un nuevo servidor
22         ServidorImpresion impresora = ServidorImpresion.ObtenerInstancia();
23         ServidorImpresion servidor = ServidorImpresion.ObtenerInstancia();
24
25         // 2 Mostrar el estado de la impresora
26         servidor.MostrarEstadoImpresora();
27     }
28     }
```

Aquí pide al usuario el nombre del documento que quiere imprimir. Cada vez que escribes algo, el programa simula enviarlo a imprimir.

```
25     // 3 bucle principal de impresión
26     while (true)
27     {
28         Console.Write("  Ingresa el nombre del documento a imprimir : ");
29         string nombre = Console.ReadLine();
30
31         if (string.IsNullOrEmpty(nombre))
32         {
33             Console.WriteLine("\n Saliendo del sistema de impresión..");
34             break;
35         }
36
37         servidor.EnviaAImprimir(nombre);
38
39         //Aquí llama al método del servidor para simular la impresión del documento.
40         //El servidor toma el documento del pool de trabajos, lo imprime y luego lo libera
41     }
42 }
```

Program

```
Program.cs  Class2.cs  Class1.cs
EXAMENU2
EXAMENU2.Program
Main(string[] args)

0 referencias
internal class Program
{
    0 referencias
    static void Main(string[] args)
    {
        Console.WriteLine("  GESTIÓN DE COLAS DE IMPRESIÓN EN RED : \n");

        // 1 Obtener la única instancia del servidor para que no crea un nuevo servidor
        ServidorImpresion impresora = ServidorImpresion.ObtenerInstancia();
        ServidorImpresion servidor = ServidorImpresion.ObtenerInstancia();

        // 2 Mostrar el estado de la impresora
        servidor.MostrarEstadoImpresora();

        // 3 bucle principal de impresión
        while (true)
        {
            Console.Write("  Ingresa el nombre del documento a imprimir : ");
            string nombre = Console.ReadLine();

            if (string.IsNullOrEmpty(nombre))
            {
                Console.WriteLine("\n Saliendo del sistema de impresión..");
                break;
            }

            servidor.EnviaAImprimir(nombre);

            //Aquí llama al método del servidor para simular la impresión del documento.
            //El servidor toma el documento del pool de trabajos, lo imprime y luego lo libera
        }
    }
}
```

PATRÓN: Singleton

La clase *ServidorImpresion* usa el patrón Singleton para que solo exista una impresora principal en toda la red, encargada de manejar los trabajos de impresión.

Además, trabaja junto con el Object Pool, que reutiliza los trabajos para hacer el sistema más rápido

```
7
8 namespace EXAMENU2
9 {
10     // PATRÓN: Singleton
11     8 referencias
12     class ServidorImpresion //Clase
13     {
14         //Atributos
15
16         private static ServidorImpresion instancia; //guardará la única instancia del servidor.
17         private static readonly object lockSingleton = new object(); //bloquear el acceso intentan crear varios hilos instancia al mismo tiempo
18
19         private readonly PoolTrabajos pool; //Lista de trabajos de impresión
20
21         // Datos del servidor
22         private readonly string modelo = "EPSON ";
23         private readonly int velocidadPPM = 50;
24         private bool estaEncendida = true;
25
26         2 referencias
27         public string IP { get; private set; } //El private solo la propia clase puede modificarlas
28         2 referencias
29         public string Subred { get; private set; }
30         2 referencias
31         public string Gateway { get; private set; }
32         2 referencias
33         public string SSID { get; private set; }
34
35         1 referencia
36         private ServidorImpresion()
37         {
38             pool = new PoolTrabajos();
39
40             IP = "192.168.1.100";
41             Subred = "255.255.255.0";
42             Gateway = "192.168.1.1";
43             SSID = "RedEscuela";
44
45             Console.ForegroundColor = ConsoleColor.Yellow;
46             Console.WriteLine(" La impresora principal está lista para imprimir.\n");
47             Console.ResetColor();
48         }
49     }
50 }
```

```

43 // Patrón Singleton
44
45 2 referencias
46 public static ServidorImpresion ObtenerInstancia()
47 {
48     if (instancia == null)
49     {
50         lock (lockSingleton) //evitar que otros hilos la creen al mismo tiempo
51         {
52             if (instancia == null)
53                 instancia = new ServidorImpresion(); //Crea la instancia una sola vez
54         }
55     }
56     return instancia; //Devuelve esa misma instancia cada vez que se llame.
57 }
58
59 //
60 //imprime en pantalla la información
61 1 referencia
62 public void MostrarEstadoImpresora()
63 {
64     Console.ForegroundColor = ConsoleColor.Blue;
65     Console.WriteLine(" ESTADO DE LA IMPRESORA ");
66     Console.ForegroundColor = ConsoleColor.DarkYellow;
67     Console.WriteLine($"Modelo: {modelo}");
68     Console.WriteLine($"Velocidad: {velocidadPPM} ppm");
69     Console.WriteLine($"Estado: {(estaEncendida ? "Encendida" : "Apagada")}");
70     Console.WriteLine($"IP: {IP}");
71     Console.WriteLine($"Subred: {Subred}");
72     Console.WriteLine($"Gateway: {Gateway}");
73     Console.WriteLine($"SSID: {SSID}");
74     Console.WriteLine("-----\n");
75     Console.ResetColor();
76 }
77
78 //simula enviar un archivo a la impresora
79 1 referencia
80 public void EnviarAImprimir(string nombreDocumento)
81 {
82     TrabajoImpresion trabajo = pool.ObtenerTrabajo(nombreDocumento);
83
84     if (trabajo != null)
85     {
86         trabajo.Imprimir();
87         pool.LiberarTrabajo(trabajo); //Si el trabajo existe, lo imprime y luego lo libera.
88     }
89     else
90     {
91         Thread.Sleep(1000); //Si no hay trabajo disponible, espera un segundo
92     }
93 }
94 }

```

PATRÓN: Object Pool

El código usa el patrón Object Pool para reutilizar trabajos de impresión en lugar de crear nuevos cada vez. La clase *TrabajoImpresion* representa cada documento, y *PoolTrabajos* gestiona una “piscina” con tres trabajos disponibles que se toman, usan e inmediatamente regresan para volver a usarse.

```

8 namespace EXAMEN02
9 {
10     // CLASE: Trabajo de Impresión (objeto reutilizable)
11     //objeto reutilizable dentro del patrón Object Pool.
12
13     6 referencias
14     class TrabajoImpresion
15     {
16         //propiedad llamada Documento para guardar el nombre del archivo
17         4 referencias
18         public string Documento { get; set; }
19
20         //método Imprimir()
21         1 referencia
22         public void Imprimir()
23         {
24             Console.ForegroundColor = ConsoleColor.Yellow;
25             Console.WriteLine($" \n IMPRIMIENDO Documento: {Documento}...");
26             Thread.Sleep(2000); // Simula impresión
27             Console.WriteLine($" {Documento} SE IMPRIMIO CORRECTAMENTE : .\n");
28             Console.ResetColor();
29         }
30     }
31 }

```

```

29 // PATRÓN: Object Pool
30 // Administra los trabajos de impresión reutilizables
31 3 referencias
32 class PoolTrabajos
33 {
34     //atributo Disponible lista que guarda los trabajos libres
35     private
36
37     List<TrabajoImpresion> disponibles = new List<TrabajoImpresion>();
38
39     //tomar trabajos al mismo tiempo.
40     private readonly object lockObject = new object();
41
42     //Definimos una capacidad máxima de 3 trabajos
43     //(no se pueden tener más a la vez).
44     private const int CAPACIDAD_MAXIMA = 3;
45
46     // 3 objetos
47
48

```

```

49 // 3 objetos
50 1 referencia
51 public PoolTrabajos()
52 {
53     for (int i = 0; i < CAPACIDAD_MAXIMA; i++)
54         disponibles.Add(new TrabajoImpresion()); //y los guarda en la lista de disponibles.
55
56     Console.ForegroundColor = ConsoleColor.Cyan;
57     Console.WriteLine($" {CAPACIDAD_MAXIMA} Trabajos Disponibles : .\n");
58     Console.ResetColor();
59 }
60
61 1 referencia
62 public TrabajoImpresion ObtenerTrabajo(string nombreDocumento) //Metodo
63 {
64     lock (lockObject) //no tomen el mismo trabajo
65     {
66         if (disponibles.Count > 0)
67         {
68             var trabajo = disponibles[0];
69             disponibles.RemoveAt(0);
70             trabajo.Documento = nombreDocumento;
71
72             Console.ForegroundColor = ConsoleColor.Green;
73             Console.WriteLine($" {nombreDocumento} 'RESTAN : {disponibles.Count}");
74             Console.ResetColor();
75
76             return trabajo;
77         }
78         else
79         {
80             return null; //Si no hay trabajos disponibles, devuelve
81         }
82     }
83 }
84

```

```

85 1 referencia
86 public void LiberarTrabajo(TrabajoImpresion trabajo)
87 {
88     lock (lockObject)
89     {
90         trabajo.Documento = null; //Limpia el nombre del documento
91         disponibles.Add(trabajo); //Lo agrega de nuevo a la lista
92
93         Console.ForegroundColor = ConsoleColor.Magenta;
94         Console.WriteLine("Trabajo devuelto ");
95         Console.WriteLine($"Total disponibles: {disponibles.Count}\n");
96         Console.ResetColor();
97     }
98 }
99
100
101

```

Concola

```
C:\Users\liczy\source\repos\E  ×  +  v
GESTIÓN DE COLAS DE IMPRESIÓN EN RED :

3 Trabajos Disponibles : .

La impresora principal está lista para imprimir.

    ESTADO DE LA IMPRESORA
Modelo: EPSON
Velocidad: 50 ppm
Estado: Encendida
IP: 192.168.1.100
Subred: 255.255.255.0
Gateway: 192.168.1.1
SSID: RedEscuela
-----

Ingresa el nombre del documento a imprimir : Doc1
'Doc1'RESTAN : 2

    IMPRIMIENDO Documento: Doc1...
Doc1 SE IMPRIMIO CORRECTAMENTE : .

Trabajo devuelto
Total disponibles: 3

Ingresa el nombre del documento a imprimir : |
```

Conclusión

Conclusión sobre la Fusión de Patrones (Singleton y Object Pool) En él la problemática utilicé una combinación de dos patrones: Singleton y Object Pool, que juntos permiten manejar la red de impresión de forma centralizada y eficiente. El Singleton (ServidorImpresión) garantiza que solo exista una única impresora principal en toda la red. Esto es importante porque evita conflictos y asegura que todos los usuarios trabajen con la misma configuración de red. En otras palabras, hay un solo punto de control que coordina todo. Por otro lado, el Object Pool (PoolTrabajos) se encarga de mejorar la velocidad y el rendimiento. En lugar de crear y eliminar trabajos de impresión cada vez, el sistema utiliza los mismos objetos, lo que ahorra tiempo y recursos. La unión de ambos patrones logra un equilibrio perfecto: el servidor único controla los recursos limitados del pool, manteniendo el sistema ordenado, rápido y evitando que se sobrecargue. Gracias a esta fusión, la gestión de impresión se vuelve más estable, eficiente y fácil de mantener.